

Impactis Project

First Milestone Report — Restaurant Ordering REST API

1. Summary

This milestone delivers a complete RESTful backend for a restaurant ordering platform, built with Spring Boot and PostgreSQL, containerized with Docker. All seven required endpoints are implemented, backed by a layered architecture (Controllers, Services, JPA Repositories), with request validation, centralized error handling, automated test coverage above 90%, and load testing performed with K6. The application was developed, tested, and validated end-to-end on a local development environment.

2. Technology Stack

Component	Technology
Language / Runtime	Java 25 (Eclipse Temurin)
Framework	Spring Boot 4.1.1 (Web, Data JPA, Validation, Security)
Build Tool	Maven (via Maven Wrapper)
Database	PostgreSQL 16 (Docker container)
Containerization	Docker Desktop / Docker Compose
Security	Spring Security (BCrypt password hashing)
Testing	JUnit 5, Mockito, AssertJ, Spring MockMvc
Coverage	JaCoCo (90% minimum line coverage enforced)
Load Testing	Grafana k6

3. Architecture & Data Model

The application follows a standard layered architecture, with clear separation between HTTP handling, business logic, and persistence:

- **Controllers** — handle HTTP requests/responses for all 7 endpoints
- **Services** — contain business logic: validation, price calculation, transactional order creation, password hashing
- **Repositories** — Spring Data JPA interfaces providing database access
- **Entities** — JPA-mapped classes for Restaurant, User, MenuItem, Order, and OrderItem
- **DTOs** — dedicated request/response objects, decoupled from database entities

Database Schema

PostgreSQL schema with 5 tables: **restaurants**, **users**, **menu_items**, **orders**, and **order_items**, with foreign key relationships linking menu items to restaurants, and orders to both users and restaurants (with **order_items** as the join between orders and menu items).

4. Implemented Endpoints

#	Method	Path	Description
1	GET	/restaurants	List all restaurants
2	GET	/restaurants/{id}	Get a single restaurant
3	POST	/restaurants	Create a restaurant
4	POST	/restaurants/{id}/menu-items	Add a menu item to a restaurant
5	GET	/menu?restaurantId=	Get menu for a restaurant
6	POST	/order	Place an order
7	POST	/users	Create a user

All endpoints were verified end-to-end using manual HTTP testing (curl) against the running application and a live PostgreSQL instance, in addition to automated tests.

5. Design Considerations Addressed

- **Robust business logic:** order totals (subtotal, delivery fee, total) are always calculated server-side from the database, never trusted from client input, preventing price manipulation.
- **Request validation:** Bean Validation annotations enforce required fields (e.g. `userId` and `restaurantId` cannot be null on orders), valid email format, and positive prices/quantities. Verified with both automated tests and manual invalid-request testing.
- **Layered separation:** validation, business logic, and persistence are kept in their respective layers (DTOs, Services, Repositories).
- **Global Exception Handler:** a centralized handler converts internal errors into clean, consistent JSON responses (404 for missing resources, 400 with field-level messages for validation failures, 500 as a safe fallback), rather than leaking raw stack traces.
- **Test coverage:** JaCoCo integrated into the Maven build, enforcing a 90% minimum line coverage threshold; the build fails automatically if coverage drops below this bar.
- **Security:** user passwords are hashed with BCrypt before storage; password/password-hash values are never present in any API response.

6. Automated Testing & Coverage

A full suite of unit and integration tests was written covering all four services (Restaurant, MenuItem, User, Order) and all four controllers, using Mockito for repository mocking and Spring's MockMvc for HTTP-layer testing.

Layer	Coverage focus
Service tests	CRUD logic, not-found handling, order total calculation, price integrity, password hashing
Controller tests	HTTP status codes, JSON response shape, validation rejection (400), not-found handling (404)

Result: Final build achieved **BUILD SUCCESS** with line coverage $\geq 90\%$, the minimum required by the milestone specification.

7. Load Testing (K6)

Grafana k6 was used to generate test data and evaluate application performance under both a sudden traffic spike and a sustained, escalating load.

7.1 Test Data Generation

A seeding script created 20 restaurants (each with 2–4 menu items), 100 users, and 150 orders via the live API, populating the database with realistic data for subsequent load tests.

Metric	Result
Total checks	324 / 324 passed (100%)
Restaurants created	20
Users created	100
Orders created	150
Failed requests	0

7.2 Burst Test

Simulated a sudden spike to 100 concurrent virtual users (ramped up in 5 seconds, held for 20 seconds), against a realistic traffic mix: browsing restaurants (40%), viewing a restaurant (30%), checking a menu (20%), and placing an order (10%).

Metric	Result	Threshold	Status
95th percentile response time	676 ms	< 1000 ms	PASS
Failed request rate	0.00%	< 5%	PASS
Total requests	5,833	—	—
Throughput	~186 req/s	—	—

Average response time	335 ms	—	—
-----------------------	--------	---------	---------

Result: Both thresholds passed. The application handled a sudden spike to 100 concurrent users with zero failed requests and response times well within acceptable limits.

7.3 Stress Test

Load was ramped gradually from 0 to 300 concurrent virtual users over five stages (50 → 100 → 200 → 300 users, 20 seconds per stage) to identify the point at which performance degrades under sustained, escalating load.

Metric	Result	Threshold	Status
95th percentile response time	2.06 s	< 2 s	THRESHOLD CROSSED
Failed request rate	0.00%	< 20%	PASS
Total requests	14,105	—	—
Peak concurrent users	300	—	—
Max response time observed	6.68 s	—	—

Result: Zero request failures even at 300 concurrent users — the application remained functionally correct under heavy load. Response times degraded beyond the 2-second target at peak load, identifying a performance ceiling rather than a functional failure. This is consistent with the known testing limitations below (single-machine setup: Docker, database, and application competing for the same CPU/RAM resources).

8. Testing Limitations

- Docker daemon resource caps limit the database container's available CPU/RAM on the local development machine.
- k6 itself requires memory to spin up virtual users, competing with the application and database for the same host resources.
- All testing was performed on a single laptop, meaning results at high concurrency (e.g. the stress test's 300 VU stage) reflect the shared-resource ceiling of one machine, not a representative multi-node production deployment.

9. Conclusion

The first milestone is complete. All seven required endpoints are implemented and manually verified, business logic is validated and unit-tested, error handling is centralized and consistent, automated test coverage exceeds the 90% requirement, and the application has been load-tested using k6 for both burst and sustained-stress scenarios. The application performs reliably under normal and spike conditions and remains functionally correct (zero errors) even under heavy sustained load, with the observed performance ceiling attributable to local single-machine resource constraints rather than application defects.

